

Fiche récapitulative – STA211

Travaux Pratiques

Deep Learning avec Keras et Manifold Untangling

Nicolas Thome (CNAM)

14 juin 2026

Résumé

Ce TP pratique la bibliothèque Keras sur MNIST : régression logistique, MLP, CNN (type LeNet), puis visualise le *manifold untangling* par t-SNE, enveloppes convexes, ellipses et Neighborhood Hit.

Table des matières

1	Partie 1 : Deep Learning avec Keras	2
1.1	Exercice 0 : Chargement des données MNIST	2
1.2	Exercice 1 : Régression logistique	2
1.3	Exercice 2 : Perceptron multi-couches (MLP)	2
1.4	Exercice 3 : Réseau de neurones convolutif (CNN)	3
2	Partie 2 : Deep Learning et Manifold Untangling	3
2.1	Exercice 4 : Visualisation avec t-SNE	3
2.2	Exercice 5 : Représentations internes des réseaux	4
	Questions clés (QCM)	4

Partie 1 : Deep Learning avec Keras

Exercice 0 : Chargement des données MNIST

MNIST : 60 000 images train, 10 000 test, 28×28 , 10 classes (chiffres 0–9).

```
1 from keras.datasets import mnist
2 (X_train, y_train), (X_test, y_test) = mnist.load_data()
3 X_train = X_train.reshape(60000, 784).astype('float32') / 255
4 X_test = X_test.reshape(10000, 784).astype('float32') / 255
```

Afficher les 200 premières images avec Matplotlib.

Question : espace des images ? $\mathbb{R}^{784}$.

Exercice 1 : Régression logistique

Modèle : $p(\hat{y}_{c,i} | \mathbf{x}_i) = \frac{e^{(\mathbf{x}_i; \mathbf{w}_c) + b_c}}{\sum_{c'} e^{(\mathbf{x}_i; \mathbf{w}_{c'}) + b_{c'}}$

Nombre de paramètres : $784 \times 10 + 10 = 7850$.

```
1 model = Sequential()
2 model.add(Dense(10, input_dim=784, name='fc1'))
3 model.add(Activation('softmax'))
```

One-hot encoding :

```
1 from keras.utils import np_utils
2 Y_train = np_utils.to_categorical(y_train, 10)
```

Entropie croisée : $\mathcal{L}_{\mathbf{w}, \mathbf{b}}(\mathcal{D}) = -\frac{1}{N} \sum_{i=1}^N \log(\hat{y}_{c^*, i})$

Convexité : oui, LR + cross-entropy = problème convexe $\Rightarrow$ minimum global assuré avec bon pas.

```
1 sgd = SGD(learning_rate=0.5)
2 model.compile(loss='categorical_crossentropy',
3               optimizer=sgd, metrics=['accuracy'])
4 model.fit(X_train, Y_train, batch_size=100,
5           epochs=10, verbose=1)
6 scores = model.evaluate(X_test, Y_test, verbose=0)
```

Résultat attendu : $\sim 92\%$ accuracy test.

Exercice 2 : Perceptron multi-couches (MLP)

Une couche cachée $L = 100$, sigmoïde, softmax en sortie.

Nombre de paramètres : $784 \times 100 + 100 + 100 \times 10 + 10 = 79510$.

```
1 model = Sequential()
2 model.add(Dense(100, input_dim=784, name='fc1'))
3 model.add(Activation('sigmoid'))
4 model.add(Dense(10, name='fc2'))
5 model.add(Activation('softmax'))
```

Non-convexité : MLP + sigmoïde = problème non convexe. Pas de garantie de minimum global.

Résultat attendu : $\sim 98\%$ accuracy test.

Exercice 3 : Réseau de neurones convolutif (CNN)

Reformatage : $28 \times 28 \times 1$.

```
1 X_train = X_train.reshape(-1, 28, 28, 1)
2 X_test  = X_test.reshape(-1, 28, 28, 1)
```

Architecture LeNet-like :

1. Conv2D(16, 5×5 , ReLU) + MaxPooling(2×2)
2. Conv2D(32, 5×5 , ReLU) + MaxPooling(2×2)
3. Flatten + Dense(100, sigmoïde) + Dense(10, softmax)

```
1 model = Sequential()
2 model.add(Conv2D(16, kernel_size=(5,5),
3                 activation='relu', input_shape=(28,28,1)))
4 model.add(MaxPooling2D(pool_size=(2,2)))
5 model.add(Conv2D(32, (5,5), activation='relu'))
6 model.add(MaxPooling2D(pool_size=(2,2)))
7 model.add(Flatten())
8 model.add(Dense(100, activation='sigmoid'))
9 model.add(Dense(10, activation='softmax'))
```

Résultat attendu : $\sim 99\%$ accuracy test.

Sauvegarde du modèle :

```
1 def saveModel(model, savename):
2     model_yaml = model.to_yaml()
3     with open(savename+".yaml", "w") as f:
4         f.write(model_yaml)
5     model.save_weights(savename+".h5")
```

Partie 2 : Deep Learning et Manifold Untangling

Exercice 4 : Visualisation avec t-SNE

t-SNE : réduction non linéaire en 2D qui préserve les proximités entre points.

```
1 from sklearn.manifold import TSNE
2 tsne = TSNE(n_components=2, init='pca',
3             perplexity=30, verbose=2)
4 X_r = tsne.fit_transform(X_test[:1000])
```

Trois métriques de séparation :

1. **Enveloppe convexe (ConvexHull)** : périmètre minimum contenant tous les points d'une classe.
2. **Ellipse de meilleure approximation** : ajustement d'une gaussienne 2D par classe (Gaussian-Mixture).
3. **Neighborhood Hit (NH)** : pour chaque point, proportion des k plus proches voisins de la même classe. Moyennée sur toute la base.

```
1 def neighboring_hit(points, labels, k=6):
2     nbrs = NearestNeighbors(n_neighbors=k+1,
3                             algorithm='ball_tree').fit(points)
4     distances, indices = nbrs.kneighbors(points)
5     txs = 0.0
6     for i in range(len(points)):
7         tx = sum(1 for j in range(1, k+1)
8                 if labels[indices[i,j]] == labels[i]) / k
9         txs += tx
10    return txs / len(points)
```

Comparaison ACP vs t-SNE : t-SNE sépare mieux les classes non linéairement séparables.

Exercice 5 : Représentations internes des réseaux

Objectif : visualiser l'effet de *manifold untangling* – les réseaux de neurones transforment un espace d'entrée peu séparable en un espace latent linéairement séparable.

Chargement du modèle :

```
1 def loadModel(savename):
2     with open(savename+".yaml", "r") as f:
3         model = model_from_yaml(f.read())
4         model.load_weights(savename+".h5")
5     return model
```

Extraction des représentations latentes :

```
1 model.pop() # supprime la couche softmax
2 model.pop() # supprime la couche FC
3 representations = model.predict(X_test)
```

Résultat : Les représentations internes (MLP caché 100D ou CNN) montrent une bien meilleure séparation des classes que l'espace d'entrée original, visible par t-SNE.

À retenir. Le *manifold untangling* est la capacité des réseaux profonds à «démêler» les variétés des données, rendant les classes linéairement séparables dans l'espace des représentations internes.

Questions clés (QCM)

1. Combien de paramètres un modèle de régression logistique a-t-il sur MNIST (28×28 , 10 classes) ?
- (a) 784
 - (b) 7 850
 - (c) 10 000
 - (d) 79 510

Réponse : b ($784 \times 10 + 10 = 7\,850$).

2. La fonction de coût de la régression logistique (cross-entropy) est-elle convexe ?
- (a) Oui, minimum global garanti
 - (b) Non, minimum local seulement
 - (c) Seulement avec SGD
 - (d) Cela dépend de l'initialisation

Réponse : a (**LR + cross-entropy = convexe**).

3. Combien de paramètres un MLP avec 1 couche cachée de 100 neurones a-t-il sur MNIST ?
- (a) 7 850
 - (b) 79 510
 - (c) 100 000
 - (d) 797 000

Réponse : b ($784 \times 100 + 100 + 100 \times 10 + 10$).

4. Quelle accuracy peut-on attendre du MLP sur MNIST ?
- (a) 85 %
 - (b) 92 %
 - (c) 98 %
 - (d) 99 %

Réponse : c (**$\sim 98\%$**).

5. Quelle accuracy peut-on attendre du CNN (LeNet-like) sur MNIST ?
- (a) 85 %
 - (b) 92 %
 - (c) 98 %
 - (d) 99 %

Réponse : d (**$\sim 99\%$**).

6. Que signifie l'acronyme t-SNE ?
- (a) *t-Statistics and Neural Embedding*
 - (b) *t-Distributed Stochastic Neighbor Embedding*
 - (c) *Topological Sample Neighborhood Estimation*
 - (d) *Tensor-Structured Nonlinear Embedding*

Réponse : b.

7. Que mesure la métrique Neighborhood Hit (NH) ?
- (a) La distance moyenne entre classes
 - (b) Le taux de voisins de même classe parmi les k plus proches voisins
 - (c) La variance intra-classe
 - (d) Le nombre de clusters

Réponse : b.

8. Quel est le principe du *manifold untangling* ?

- (a) Réduire la dimension des données
- (b) Démêler les variétés pour rendre les classes séparables dans l'espace latent
- (c) Augmenter le nombre de paramètres
- (d) Initialiser les poids aléatoirement

Réponse : b.

9. Comment extraire la couche cachée d'un modèle Keras ?

- (a) `model.predict(layer=hidden)`
- (b) `model.pop()` puis `model.predict()`
- (c) `model.get_layer('hidden').output`
- (d) `model.forward(hidden=True)`

Réponse : b ou c (pop + predict ou get_layer).

10. Quelle méthode de visualisation non linéaire est utilisée dans le TP ?

- (a) ACP
- (b) t-SNE
- (c) LDA
- (d) k-means

Réponse : b (t-SNE).

Références

- [1] F. Chollet. *Keras*. <https://keras.io>, 2015.
- [2] L. van der Maaten, G. Hinton. *Visualizing Data using t-SNE*. JMLR, 9:2579–2605, 2008.
- [3] Y. LeCun et al. *Backpropagation applied to handwritten zip code recognition*. Neural Computation, 1(4):541–551, 1989.